

به نام خدا

پروژه ی نهایی هوش محاسباتی: طراحی سیستم ترافیکی هوشمند برای چهارراه

ایلیا جاهد 402521126

عرفان قاسمیان 402521495

در ابتدا یک نکته در مورد پروژه ذکر میشود:

این داکيومنت سعی میشود متمرکز بر بخش experiment ها شود بر این اساس تمامی توضیحات از فاز 1 گرفته تا فاز 4 تماما در پروژه در بخش های مربوطه ضمن comment گذاری کد (با ai) ، فایل های ریدمی کامل و دقیق قرار داده شده است تا در صورت نیاز از آن منابع نیست استفاده شود.ضمن اینکه با توجه به اینکه این پروژه در گیت هاب قرار داده شده است تمامی کامنت ها و فایل های ریدمی به زبان انگلیسی است ولی در این داک سعی میشود مفاهیم و مقدمات اولیه و همچنین آنالیز نهایی تماما فارسی مناسب ارائه تهیه گردد.

ادرس گیت هاب جهت بررسی های بیشتر در صورت علاقه مندی :

<https://github.com/IlyaJahed2004/smart-traffic-light-controller>

در این پروژه سعی کردیم تمامی پارامتر های به صورت واقعی initialize شود (بعنوان مثال سعی شده بر اساس cycle time فاز ی کنترلر صدا زده شود.یعنی قرار نیست در هر تیک زمانی به طور مکرر و با فرکانس بالا چراغ سبز و قرمز داشته باشیم.بقیه ی موارد هم سعی میشود در ادامه در همین داکيومنت ذکر شود و در صورت نیاز به بررسی بیشتر ادرس گیت هاب چک شود).

پروژه‌ی حاضر به طراحی و پیاده‌سازی یک سیستم کنترل هوشمند چراغ راهنمایی در یک تقاطع دو-جاده‌ای می‌پردازد که با استفاده از منطق فازی (Fuzzy Logic) تصمیم می‌گیرد و پارامترهای این سیستم فازی توسط دو الگوریتم فراابتکاری (متهیوریستیک) بهینه‌سازی ازدحام ذرات (PSO) و بهینه‌سازی کلونی مورچگان (ACO) تنظیم و بهینه می‌شوند.

۱. تعریف مسئله

یک تقاطع فرضی با دو جاده در نظر گرفته شده که خودروها به صورت تصادفی (فرآیند پواسون) در هر جاده وارد می‌شوند و در صف می‌مانند تا نوبت عبورشان برسد. هدف، طراحی یک کنترل‌کننده‌ی هوشمند است که بر اساس طول صف هر جاده، مدت‌زمان چراغ سبز هر جاده را به صورت پویا تعیین کند، به گونه‌ای که معیار هزینه‌ی زیر کمینه شود:

$$C = \alpha \cdot W + \beta \cdot Q + \gamma \cdot S$$

که در آن W میانگین زمان انتظار خودروها، Q میانگین طول صف و S تعداد توقف‌های خودروهاست.

۲. معماری کلی پروژه (چهار فاز)

فاز ۱ — شبیه‌سازی ترافیک (TrafficEnv) یک محیط شبیه‌سازی گسسته-زمانی که ورود خودروها، صف‌بندی، و خروج آن‌ها را مدل می‌کند. این فاز خودش تصمیمی نمی‌گیرد؛ فقط دنیایی می‌سازد که کنترل‌کننده‌ها در آن آزمایش شوند.

فاز ۲ — کنترل‌کننده‌ی فازی (FuzzyController) یک کنترل‌کننده‌ی فازی از نوع Mamdani با دو ورودی (طول صف هر جاده، هر کدام با سه مجموعه‌ی فازی (Low/Medium/High و یک خروجی) مدت زمان سبز جاده‌ی اول، با سه مجموعه‌ی (Short/Medium/Long)، به همراه ۹ قانون فازی. تمام پارامترهای قابل تنظیم (شکل توابع عضویت ورودی/خروجی و وزن قوانین) در یک بردار مسطح ۲۷ عددی بسته‌بندی شده‌اند تا مستقیماً در اختیار الگوریتم‌های بهینه‌سازی فاز ۳ قرار گیرند.

فاز ۳ — بهینه‌سازی پارامترها: (ACO و PSO) دو الگوریتم فراابتکاری مستقل، هر دو روی همان بردار ۲۷ عددی فاز ۲ کار می‌کنند و هدفشان کمینه کردن تابع هزینه بالاست:

- PSO: پیاده‌سازی استاندارد ازدحام ذرات (global-best) با اینرسی کاهش و مکانیزم seeding اختیاری که یک ذره را دقیقاً از بردار پیش‌فرض دستی (baseline) شروع می‌دهد.

- ACO: نسخه‌ی گسسته‌شده‌ی الگوریتم کلاسیک کلونی مورچگان؛ هر بعد به چند سطح گسسته (bin) تقسیم شده و یک جدول فرومون واقعی روی این بین‌ها نگهداری و به‌روزرسانی می‌شود.

فاز ۴ — مقایسه‌ی الگوریتم‌ها: نتایج PSO و ACO از چهار منظر مقایسه می‌شوند: (۱) مقدار نهایی تابع هزینه، (۲) سرعت همگرایی، (۳) پایداری جواب در اجراهای مستقل مختلف، و (۴) تأثیر واقعی بهینه‌سازی روی عملکرد کنترل‌کننده در سناریوهای مختلف ترافیکی.

۳. رویکرد ارزیابی (Fitness) و استحکام (Robustness)

برای جلوگیری از overfit شدن پارامترهای بهینه‌شده روی یک الگوی ترافیکی خاص، ارزیابی هر کاندید (چه ذره در PSO، چه مورچه در ACO) بر اساس میانگین هزینه روی چند سناریوی ترافیکی مختلف انجام می‌شود، نه یک سناریوی ثابت. این سناریوها ترکیبی از چند الگوی نرخ ورود خودرو (متقارن/نامتقارن، سبک/سنگین) و چند seed مستقل برای هر الگو هستند، تا هم تنوع کافی از شرایط ترافیکی پوشش داده شود و هم جواب نهایی به یک دنباله‌ی رندوم خاص وابسته نباشد.

۴. معیار مقایسه با نتیجه‌ی پایه (Baseline)

نتیجه‌ی پارامترهای دستی تنظیم‌شده‌ی فاز ۲ (بدون هیچ بهینه‌سازی) به‌عنوان baseline در نظر گرفته شده و تمام نتایج PSO و ACO نسبت به آن سنجیده می‌شوند. برای اطمینان از صحت این مقایسه، امکان شروع از همان بردار پایه برای هر دو الگوریتم فراهم شده است؛ در حالت PSO این امر یک تضمین ریاضی قطعی ایجاد می‌کند (جواب نهایی هرگز بدتر از خط مبنا نخواهد بود)، در حالی که در ACO تنها یک soft bias است، نه یک تضمین قطعی.

در این بخش سعی شده در مورد فازهای مختلف پروژه توضیحی کافی ارائه شود تا درک بخش Experiments و تحلیل آن‌ها ساده‌تر گردد. در ریشه‌ی پروژه یک README برای معرفی کلی تمام فازها، و به‌صورت جزئی‌تر در فولدر اختصاصی هر فاز، یک README دقیق و کامل برای همان فاز قرار داده شده است. در صورت نیاز به بررسی بیشتر، می‌توان به آدرس گیت‌هاب پروژه مراجعه کرد.

فاز ۱: شبیه‌سازی ترافیک (TrafficEnv)

این فاز، زیرساخت پایه‌ی کل پروژه است و هیچ تصمیمی درباره‌ی زمان‌بندی چراغ نمی‌گیرد؛ فقط یک «دنیای آزمایشی» می‌سازد که فازهای بعدی در آن سنجیده می‌شوند.

- **ورود خودروها:** در هر تیک زمانی، تعداد خودروهای واردشده به هر جاده از یک فرآیند پواسون با نرخ قابل تنظیم ($arrival_rate_1, arrival_rate_2$) تولید می‌شود — یعنی نه هر تیک دقیقاً یک خودرو، بلکه یک میانگین تصادفی و واقع‌گرایانه.
- **خروج خودروها:** فقط جاده‌ای که چراغش سبز است اجازه‌ی تخلیه دارد، و آن هم حداکثر به تعداد $departure_rate$ خودرو در هر تیک — یعنی چراغ سبز، صف را یک‌باره خالی نمی‌کند.
- **معیارهای خروجی:** میانگین زمان انتظار (W)، میانگین طول صف (Q)، و تعداد توقف‌ها — (S) دقیقاً همان سه متغیری که بعداً در تابع هزینه $C = \alpha \cdot W + \beta \cdot Q + \gamma \cdot S$ استفاده می‌شوند.

- برای اعتبارسنجی این فاز، یک کنترل‌کننده‌ی ثابت و ساده (تعویض چراغ هر ۱۰ تیک) به‌عنوان کنترل‌کننده‌ی موقتی استفاده شد، تا پیش از جایگزینی با کنترل‌کننده‌ی هوشمند فاز ۲، مطمئن شویم شبیه‌سازی بدون خطا و با اعداد منطقی اجرا می‌شود.

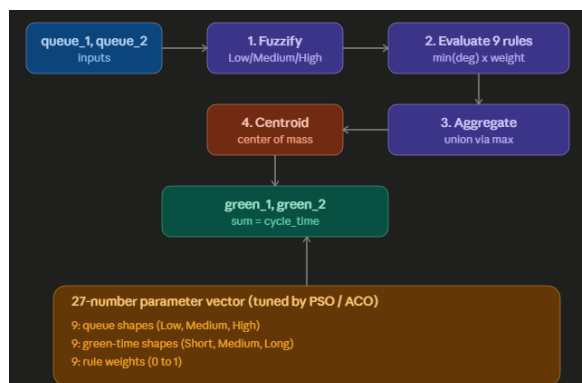
فاز ۲: کنترل‌کننده‌ی فازی (FuzzyController)

این فاز، کنترل‌کننده‌ی ثابت فاز ۱ را با یک تصمیم‌گیرنده‌ی هوشمند فازی جایگزین می‌کند که بر اساس طول صف هر دو جاده، مدت زمان چراغ سبز را تعیین می‌کند.

ورودی‌ها و خروجی: دو ورودی — طول صف جاده‌ی ۱ و جاده‌ی ۲ — هر کدام با سه دسته‌ی فازی Low/Medium/High، و یک خروجی — مدت سبز جاده‌ی ۱ — با سه دسته‌ی Short/Medium/Long. مدت سبز جاده‌ی ۲ نیز از یک بودجه‌ی زمانی ثابت (cycle_time) کسر می‌شود.

پایپ‌لاین استنتاج:

به این دیاگرام دقت شود:



دیاگرام بالا مسیر کاملی را نشان می‌دهد که در تابع `compute_green_time()` طی می‌شود: از لحظه‌ای که طول دو صف به کنترل‌کننده داده می‌شود، تا لحظه‌ای که دو عدد قطعی (مدت زمان سبز هر جاده) از آن خارج می‌شود.

- **ورودی‌ها (queue_1, queue_2):** طول فعلی صف هر دو جاده، به‌صورت دو عدد ساده، از شبیه‌سازی خوانده می‌شود.
- **گام ۱ — فازی‌سازی (Fuzzify):** هر یک از این دو عدد، به‌جای آن که فقط «کم» یا «زیاد» طبقه‌بندی شود، هم‌زمان درجه‌ی عضویتی در هر سه دسته‌ی Low، Medium، و High می‌گیرد (عددی بین صفر تا یک برای هر کدام، بر اساس توابع عضویت مثلثی). برای مثال، یک صف با طول ۱۲ ممکن است ۲۰٪ عضو Medium و ۸۰٪ عضو High باشد — همین هم‌پوشانی نرم است که باعث می‌شود تصمیم‌گیری در مرز دو دسته به‌صورت ناگهانی جهش نکند.

- **گام ۲ — ارزیابی ۹ قانون (Evaluate 9 rules):** کنترل‌کننده ۹ قانون فازی دارد که تمام ترکیب‌های ممکن دو ورودی (سه دسته $\times$ سه دسته) را پوشش می‌دهند؛ مثلاً «اگر جاده‌ی ۱ پرترافیک و جاده‌ی ۲ خلوت باشد، سبز جاده‌ی ۱ باید طولانی باشد». قدرت فعال شدن هر قانون از ترکیب دو عملگر به‌دست می‌آید: ابتدا مینیمم درجه‌ی عضویت دو ورودی مربوط به آن قانون (عملگر فازی AND)، سپس ضرب در وزن اختصاصی همان قانون — همین وزن‌هاست که توسط PSO/ACO در فاز ۳ تنظیم می‌شود.

- **گام ۳ — تجميع (Aggregate):** خروجی هر قانون به یکی از سه دسته‌ی خروجی (Short)، Medium، Long اشاره می‌کند. برای هر دسته‌ی خروجی، قوی‌ترین قانونی که به آن اشاره کرده انتخاب می‌شود (عملگر فازی OR، به‌صورت ماکسیمم). نتیجه‌ی این مرحله، یک شکل فازی ترکیبی واحد است که نمایانگر «قضاوت جمعی» تمام قوانین است.

- **گام ۴ — غیرفازی‌سازی به روش مرکز ثقل (Centroid):** این شکل فازی ترکیبی، که هنوز یک عدد قطعی نیست، به روش مرکز ثقل (center of mass) به یک عدد واحد تبدیل می‌شود — همان مدت‌زمان سبز جاده‌ی اول، بر حسب ثانیه. این دقیقاً همان روش کلاسیک Mamdani است که در اسلایدهای درس نیز آمده و طبق نیازمندی پروژه استفاده شده است.

- **خروجی‌ها (green_1, green_2):** مدت سبز جاده‌ی اول همان عددی است که از مرحله‌ی قبل به‌دست آمد. مدت سبز جاده‌ی دوم به‌جای آن که خودش فازی محاسبه شود، از بودجه‌ی زمانی ثابت `cycle_time` کسر می‌شود؛ یعنی همیشه `green_1 + green_2 = cycle_time` برقرار است — دقیقاً مانند یک چراغ راهنمایی واقعی که زمان کل هر چرخه محدود است و باید بین دو جاده تقسیم شود.

قابلیت تنظیم (بردار ۲۷ عددی): تمام اعدادی که شکل این پایپ‌لاین را تعیین می‌کنند — مختصات ۹ تابع عضویت ورودی، مختصات ۹ تابع عضویت خروجی، و ۹ وزن قانون — به‌جای پراکنده‌بودن، در قالب یک بردار مسطح و واحد از ۲۷ عدد نگهداری می‌شوند (جعبه‌ی پایین دیاگرام). همین ساختار ساده است که در فاز ۳ به الگوریتم‌های PSO و ACO اجازه می‌دهد بدون هیچ آگاهی از منطق فازی، صرفاً با تغییر این ۲۷ عدد، کل رفتار کنترل‌کننده را جست‌وجو و بهینه کنند.

در طول تست این فاز، سه اشکال واقعی (نه فرضی) پیدا و رفع شد: رفتار نادرست توابع عضویت در نقاط مرزی، فلیکرزدن غیرواقعی چراغ به‌دلیل تصمیم‌گیری در هر تیک به‌جای هر بلوک زمانی، و ریسک ناسازگاری بین `cycle_time` و محدوده‌ی زمان سبز. جزئیات کامل هر کدام در README همین فاز مستند شده است.

ارزیابی صادقانه‌ی این کنترل‌کننده با پارامترهای دستی نشان داد که همیشه بهتر از یک تایمر ثابت ساده نیست — دقیقاً همین نکته انگیزه‌ی فاز ۳ (بهینه‌سازی خودکار پارامترها) شد.

فاز ۳: بهینه‌سازی پارامترها با PSO و ACO

این فاز، به جای تنظیم دستی ۲۷ پارامتر کنترل‌کننده‌ی فازی، آن‌ها را با دو الگوریتم فراابتکاری مستقل جست‌وجو و بهینه می‌کند. هر دو الگوریتم، کنترل‌کننده و شبیه‌سازی را به صورت جعبه سیاه می‌بینند و فقط از طریق یک قرارداد مشترک با آن‌ها کار می‌کنند (`get_param_bounds()`، `set_params_from_vector()`، و یک تابع هزینه که باید کمینه شود).

الگوریتم بهینه‌سازی ازدحام ذرات (PSO):

در این پژوهش، از الگوریتم بهینه‌سازی ازدحام ذرات (Particle Swarm Optimization - PSO) برای تنظیم پارامترهای کنترل‌کننده فازی استفاده شده است. هدف این الگوریتم، یافتن مجموعه‌ای از پارامترها است که مقدار تابع هزینه سیستم کنترل ترافیک را حداقل کند. از آنجا که PSO یک روش بهینه‌سازی مبتنی بر جمعیت است، به جای جستجوی یک پاسخ منفرد، مجموعه‌ای از پاسخ‌های کاندید (ذرات) را به طور هم‌زمان در فضای جستجو حرکت می‌دهد. هر ذره در طول فرآیند بهینه‌سازی با استفاده از تجربه خود و همچنین بهترین پاسخ یافت‌شده توسط کل جمعیت، مسیر حرکت خود را اصلاح می‌کند تا به مرور به سمت نواحی با کیفیت‌تر فضای جستجو هدایت شود.

در این پیاده‌سازی، الگوریتم هیچ اطلاعی از ساختار داخلی کنترل‌کننده فازی یا شبیه‌ساز ترافیک ندارد و هر دو به صورت جعبه سیاه (Black Box) در نظر گرفته شده‌اند. بنابراین تنها ورودی الگوریتم، بردار پارامترهای کنترل‌کننده و تنها خروجی آن مقدار تابع هزینه حاصل از اجرای شبیه‌سازی است. این جداسازی باعث می‌شود الگوریتم بهینه‌سازی کاملاً مستقل از پیاده‌سازی سیستم فازی بوده و در صورت تغییر کنترل‌کننده یا تابع هزینه، نیازی به تغییر در ساختار PSO وجود نداشته باشد.

بردار هر ذره شامل ۲۷ پارامتر است که یک کنترل‌کننده فازی کامل را توصیف می‌کند. هجده پارامتر نخست مربوط به توابع عضویت هستند. سه تابع عضویت برای متغیر ورودی طول صف (Low)، Medium و High و سه تابع عضویت برای متغیر خروجی زمان چراغ سبز (Short)، Medium و Long در نظر گرفته شده است که هر یک توسط سه نقطه a ، b و c تعریف می‌شوند. نه پارامتر باقی‌مانده نیز وزن قوانین فازی را تشکیل می‌دهند.

تابع برازندگی

برای ارزیابی کیفیت هر ذره، از مفهوم تابع برازندگی (Fitness Function) استفاده شده است. در این پروژه دو تابع برای تولید تابع برازندگی پیاده‌سازی شده است. تابع `make_single_scenario_fitness` کنترل‌کننده را تنها در یک سناریوی ترافیکی اجرا کرده و مقدار تابع هزینه را بازمی‌گرداند، در حالی که تابع `make_multi_scenario_fitness` کنترل‌کننده را در چندین سناریوی مختلف اجرا کرده و میانگین (یا هر تابع تجمیع دیگر) هزینه‌ها را به عنوان مقدار برازندگی محاسبه می‌کند. به این ترتیب، الگوریتم PSO مستقل از تعداد سناریوهای مورد استفاده باقی می‌ماند و تنها یک تابع ارزیابی دریافت می‌کند.

نمایش ذرات

هر پاسخ کاندید توسط کلاس **Particle** نمایش داده می‌شود. هر ذره شامل چهار بخش اصلی است: موقعیت فعلی، سرعت، بهترین موقعیت شخصی و مقدار تابع هزینه متناظر با بهترین موقعیت شخصی. موقعیت همان بردار ۲۷ پارامتری کنترل‌کننده است و سرعت میزان تغییر هر پارامتر را در تکرار بعدی مشخص می‌کند. علاوه بر این، هر ذره بهترین پاسخ به دست آمده در طول حرکت خود را نیز ذخیره می‌کند تا بتواند در مراحل بعدی از تجربه موفق گذشته خود استفاده کند.

مقداردهی اولیه جمعیت

ایجاد جمعیت اولیه توسط تابع **initialize_population** انجام می‌شود. در این مرحله موقعیت ذرات به صورت تصادفی و یکنواخت در محدوده مجاز هر پارامتر تولید می‌شود. همچنین امکان قرار دادن یکی از ذرات دقیقاً روی بردار اولیه کنترل‌کننده نیز وجود دارد. این قابلیت با پارامتر **seed_with_default_vector** فعال می‌شود و باعث می‌شود الگوریتم علاوه بر نمونه‌های تصادفی، یک پاسخ معتبر اولیه را نیز در جمعیت داشته باشد. پس از ایجاد هر ذره، سرعت اولیه آن نیز به صورت تصادفی مقداردهی شده و سپس تابع هزینه آن محاسبه می‌شود تا بهترین موقعیت شخصی و بهترین موقعیت سراسری مقداردهی اولیه شوند.

ارزیابی هر ذره

تنها نقطه ارتباط الگوریتم **PSO** با کنترل‌کننده فازی در تابع **evaluate** قرار دارد. در این تابع، ابتدا بردار پارامترهای ذره روی کنترل‌کننده اعمال می‌شود و سپس کنترل‌کننده در محیط شبیه‌سازی اجرا شده و مقدار تابع هزینه محاسبه می‌شود. به دلیل استفاده از این ساختار، الگوریتم هیچ وابستگی مستقیمی به نحوه پیاده‌سازی سیستم فازی یا محیط شبیه‌سازی ندارد و تنها مقدار هزینه حاصل از هر پاسخ را دریافت می‌کند.

اصلاح پاسخ‌های نامعتبر

از آنجا که پارامترهای کنترل‌کننده شامل نقاط توابع عضویت مثلثی هستند، باید همواره شرط

$$a \leq b \leq c$$

برای هر تابع عضویت برقرار باشد. علاوه بر این، تمامی پارامترها باید در محدوده مجاز تعریف‌شده قرار گیرند. این وظیفه توسط تابع **repair** انجام می‌شود. در این تابع ابتدا تمام پارامترها در بازه مجاز خود محدود (**Clip**) می‌شوند. سپس برای هر یک از شش تابع عضویت مثلثی، سه نقطه تشکیل‌دهنده آن به ترتیب صعودی مرتب می‌شوند تا شکل هندسی تابع عضویت معتبر باقی بماند. از آنجا که سه نقطه هر تابع عضویت دارای محدوده یکسان هستند، این مرتب‌سازی هیچ پارامتری را از محدوده مجاز خارج نمی‌کند. وزن قوانین فازی تنها به بازه $[0, 1]$ محدود شده و نیازی به مرتب‌سازی ندارند.

به‌روزرسانی سرعت و موقعیت

اصلی‌ترین بخش الگوریتم در توابع **update_velocity** و **update_position** پیاده‌سازی شده است. در هر تکرار، ابتدا سرعت هر ذره بر اساس رابطه استاندارد **PSO** محاسبه می‌شود. سرعت جدید از سه مؤلفه تشکیل شده است. مؤلفه اول، اینرسی

است که بخشی از سرعت قبلی ذره را حفظ می‌کند و موجب ادامه حرکت در همان جهت می‌شود. مؤلفه دوم، مؤلفه شناختی است که ذره را به سمت بهترین موقعیت شخصی خود هدایت می‌کند. مؤلفه سوم نیز مؤلفه اجتماعی است که ذره را به سمت بهترین پاسخ یافت‌شده توسط کل جمعیت حرکت می‌دهد. در این پیاده‌سازی، برای جلوگیری از تغییرات بسیار بزرگ، مقدار سرعت هر بعد به یک مقدار بیشینه محدود شده است.

پس از محاسبه سرعت، موقعیت جدید ذره با جمع کردن بردار سرعت و موقعیت فعلی به دست می‌آید. سپس تابع **repair** فراخوانی می‌شود تا در صورت خروج پارامترها از محدوده مجاز یا نقض ترتیب نقاط توابع عضویت، بردار جدید اصلاح شده و همچنان یک کنترل‌کننده فازی معتبر باقی بماند.

به‌روزرسانی بهترین پاسخ‌ها

پس از ارزیابی هر ذره، ابتدا تابع **update_personal_best** بررسی می‌کند که آیا پاسخ جدید نسبت به بهترین پاسخ قبلی همان ذره بهتر است یا خیر. در صورت کاهش مقدار تابع هزینه، بهترین موقعیت شخصی و مقدار هزینه متناظر با آن به‌روزرسانی می‌شود. سپس تابع **update_global_best** بهترین پاسخ تمامی ذرات را بررسی کرده و در صورت یافتن پاسخ بهتر، بهترین پاسخ کل جمعیت را جایگزین می‌کند. این فرآیند باعث می‌شود تمام ذرات از بهترین تجربه به دست آمده توسط جمعیت بهره‌مند شوند و روند همگرایی با سرعت بیشتری انجام شود.

حلقه اصلی الگوریتم

فرآیند کامل بهینه‌سازی در تابع **optimize** پیاده‌سازی شده است. این تابع ابتدا جمعیت اولیه را ایجاد و ارزیابی می‌کند. سپس در هر تکرار، مقدار ضریب اینرسی محاسبه شده و برای تمامی ذرات، مراحل به‌روزرسانی سرعت، به‌روزرسانی موقعیت، ارزیابی تابع هزینه، به‌روزرسانی بهترین پاسخ شخصی و در نهایت به‌روزرسانی بهترین پاسخ سراسری انجام می‌شود. علاوه بر این، بهترین مقدار تابع هزینه پس از هر تکرار در متغیر **history** ذخیره می‌شود تا امکان رسم نمودار همگرایی و تحلیل روند عملکرد الگوریتم فراهم شود. در پایان، بهترین بردار ۲۷ پارامتری و مقدار تابع هزینه متناظر با آن به عنوان خروجی الگوریتم بازگردانده می‌شود.

الگوریتم بهینه‌سازی کلونی مورچگان (ACO):

الگوریتم بهینه‌سازی کلونی مورچگان (Ant Colony Optimization) یا (ACO) یکی دیگر از روش‌های فراابتکاری مبتنی بر جمعیت است که در این پژوهش برای تنظیم پارامترهای کنترل‌کننده فازی مورد استفاده قرار گرفته است. ایده اصلی این الگوریتم از رفتار مورچه‌های واقعی در یافتن کوتاه‌ترین مسیر بین لانه و منبع غذا الهام گرفته شده است. مورچه‌ها هنگام حرکت، روی مسیر خود ماده‌ای به نام فرومون ترشح می‌کنند. هرچه یک مسیر توسط مورچه‌های بیشتری استفاده شود، میزان فرومون آن افزایش یافته و در نتیجه احتمال انتخاب همان مسیر توسط سایر مورچه‌ها بیشتر می‌شود. از طرف دیگر، با گذشت زمان فرومون موجود روی مسیرها تبخیر می‌شود؛ بنابراین تنها مسیرهایی که به طور مداوم توسط مورچه‌های موفق انتخاب می‌شوند، مقدار زیادی فرومون خواهند داشت. این تعامل میان تقویت مسیرهای مناسب و تبخیر مسیرهای قدیمی باعث می‌شود کلونی به تدریج به سمت پاسخ‌های بهتر همگرا شود.

از آنجا که الگوریتم کلاسیک ACO برای مسائل گسسته طراحی شده است، امکان استفاده مستقیم از آن برای مسئله حاضر وجود ندارد؛ زیرا پارامترهای کنترل کننده فازی مقادیر پیوسته هستند. به همین دلیل، در این پژوهش ابتدا فضای جستجو گسسته سازی شده است. هر یک از ۲۷ پارامتر کنترل کننده در بازه مجاز خود به تعداد مشخصی سطح مساوی تقسیم می شود. در این پیاده سازی برای هر پارامتر ۲۰ سطح گسسته در نظر گرفته شده است؛ بنابراین هر مورچه هنگام ساخت یک پاسخ، به جای انتخاب یک مقدار حقیقی، تنها یکی از این ۲۰ سطح را انتخاب می کند. پس از انتخاب تمام سطوح، مراکز این بازه ها به عنوان مقادیر واقعی پارامترها در نظر گرفته می شوند. این روش امکان استفاده از الگوریتم کلاسیک Ant System را بدون تغییر در اصول آن فراهم می کند.

همانند الگوریتم PSO، کنترل کننده فازی و محیط شبیه سازی در این پیاده سازی به صورت یک جعبه سیاه در نظر گرفته شده اند. الگوریتم هیچ اطلاعی از ساختار داخلی سیستم فازی یا نحوه محاسبه تابع هزینه ندارد و تنها از طریق توابع عمومی کنترل کننده با سیستم ارتباط برقرار می کند. در هر ارزیابی، بردار پارامترهای تولید شده روی کنترل کننده اعمال شده و سپس مقدار تابع هزینه از طریق اجرای شبیه سازی محاسبه می شود. به همین دلیل، الگوریتم بهینه سازی کاملاً مستقل از ساختار کنترل کننده بوده و در صورت تغییر کنترل کننده یا تابع هزینه، نیازی به تغییر در ساختار ACO وجود نخواهد داشت.

هر پاسخ تولید شده توسط یک مورچه شامل همان بردار ۲۷ پارامتری کنترل کننده فازی است. هجده پارامتر نخست مربوط به شش تابع عضویت مثلثی هستند که سه تابع برای متغیر طول صف و سه تابع برای زمان چراغ سبز را شامل می شوند و هر تابع عضویت توسط سه نقطه a ، b و c تعریف می شود. نه پارامتر باقی مانده نیز وزن قوانین فازی را مشخص می کنند. علاوه بر بردار پارامترها، هر مورچه شماره سطح انتخاب شده برای هر پارامتر را نیز ذخیره می کند، زیرا این اطلاعات در مرحله به روزرسانی فرومون مورد نیاز است.

تابع `construct_ant` مسئول ساخت پاسخ های جدید است. در این تابع، مورچه پارامترهای کنترل کننده را به صورت مستقل و یک به یک انتخاب می کند. برای هر پارامتر، احتمال انتخاب هر سطح متناسب با مقدار فرومون موجود روی آن سطح محاسبه شده و سپس با استفاده از روش Roulette Wheel Selection یکی از سطوح انتخاب می شود. پس از انتخاب تمام پارامترها، بردار حاصل توسط تابع `repair` اصلاح می شود تا تمامی قیود مسئله رعایت شوند. در این مرحله، ابتدا مقادیر خارج از محدوده مجاز اصلاح شده و سپس برای هر تابع عضویت مثلثی، سه نقطه تشکیل دهنده آن مرتب می شوند تا شرط $a \leq b \leq c$ برقرار باشد. این مرحله باعث می شود تمام پاسخ های تولید شده همواره متناظر با یک کنترل کننده فازی معتبر باشند.

مهم ترین بخش الگوریتم در تابع `update_pheromone` پیاده سازی شده است که وظیفه به روزرسانی جدول فرومون را بر عهده دارد. در ابتدای هر تکرار، مقدار فرومون تمام خانه های جدول با نرخ تبخیر ρ کاهش می یابد تا تأثیر اطلاعات قدیمی به مرور کمتر شود و الگوریتم از گرفتار شدن در بهینه های محلی جلوگیری کند. پس از تبخیر، هر مورچه متناسب با کیفیت پاسخ خود روی مسیر انتخاب شده فرومون جدید اضافه می کند. مقدار این فرومون از رابطه کلاسیک Ant System یعنی

$$\Delta\tau = \frac{Q}{Cost}$$

محاسبه می‌شود؛ بنابراین هرچه مقدار تابع هزینه کمتر باشد، فرومون بیشتری روی مسیر متناظر ذخیره خواهد شد و احتمال انتخاب آن در تکرارهای بعد افزایش می‌یابد.

در این پیاده‌سازی علاوه بر مکانیزم استاندارد **Ant System**، از راهبرد تقویت نخبگان (**Elitist Reinforcement**) نیز استفاده شده است. پس از به‌روزرسانی معمول فرومون‌ها، بهترین پاسخ یافت‌شده در کل فرآیند بهینه‌سازی یک بار دیگر فرومون اضافی دریافت می‌کند. این کار باعث می‌شود اطلاعات مربوط به بهترین پاسخ موجود با سرعت کمتری از بین برود و همگرایی الگوریتم به سمت نواحی امیدبخش فضای جستجو تسریع شود.

فرآیند کلی الگوریتم در تابع **optimize** پیاده‌سازی شده است. در ابتدای اجرا، یک مجموعه اولیه از پاسخ‌ها به صورت تصادفی تولید می‌شود تا جدول فرومون مقداردهی اولیه مناسبی پیدا کند. این مرحله نقش نمونه‌برداری اولیه از فضای جستجو را ایفا می‌کند. سپس در هر تکرار، مورچه‌های جدید بر اساس جدول فرومون فعلی پاسخ‌های جدیدی تولید می‌کنند، این پاسخ‌ها توسط شبیه‌ساز ارزیابی می‌شوند، بهترین پاسخ سراسری در صورت نیاز به‌روزرسانی شده و در نهایت جدول فرومون با اعمال تبخیر، افزودن فرومون مورچه‌ها و تقویت پاسخ نخبه اصلاح می‌شود. این فرآیند تا رسیدن به تعداد تکرارهای تعیین‌شده ادامه یافته و در پایان، بهترین بردار ۲۷ پارامتری و مقدار تابع هزینه متناظر با آن به عنوان خروجی الگوریتم بازگردانده می‌شود.

از آنجا که در این پژوهش دقیقاً همان تابع هزینه و همان کنترل‌کننده فازی برای هر دو الگوریتم **PSO** و **ACO** استفاده شده است، تفاوت نتایج نهایی صرفاً ناشی از تفاوت در نحوه جستجوی فضای پاسخ توسط این دو الگوریتم است و مقایسه عملکرد آن‌ها کاملاً منصفانه خواهد بود.

رسیدیم به اصلی ترین فاز که این قسمت قرار است به صورت مفصل بررسی بشود.

فاز ۴: مقایسه‌ی الگوریتم‌ها

به فایل مربوط به **compare_optimizers.py** توجه شود. به‌طور خلاصه این فایل جایی است که **ACO** و **PSO** رو (که) هرکدام در فاز سوم ساخته و تست شده اند. این بخش کاملاً کامنت گذاری شده است و میتوانید در صورت نیاز به راحتی اجرا کرده و نتایج را مشاهده کنید) کنار هم می‌ذاریم و طبق ۴ معیار مشخص‌شده در مستندات پروژه مقایسه می‌کنیم. این اسکریپت هیچ منطق جدیدی از **PSO/ACO** نمی‌سازد. صرفاً اسکریپت‌های **run_pso_optimization.py** و **run_aco_optimization.py** را با یک تنظیمات مشترک صدا می‌زنیم تا مقایسه منصفانه باشه.

در بخش اول این فایل در حقیقت یک محیط آزمایشی یکسان برای هر دو الگوریتم داریم. سناریو ها و **seed** های مد نظرمان به این شرح است:

```
SCENARIOS = [ ("moderate symmetric", 0.3, 0.3), ("moderate asymmetric", 0.4, 0.2),  
("heavy asymmetric", 0.6, 0.2), ("heavy symmetric", 0.5, 0.5), ]
```

```
SEEDS = [1, 2, 3]
```

یعنی ۴ سناریوی ترافیکی مختلف (سبک/سنگین، متوازن/نامتوازن) ضرب در ۳ تا seed تصادفی مختلف میشود ۱۲ ترکیب سناریو / seed) هر کاندید چه ذره‌ی PSO، چه مورچه‌ی ACO روی هر ۱۲ تا این ترکیب‌ها اجرا و ارزیابی میشوند، و میانگین هزینه اش گرفته میشود. این دقیقا همان کاری است که در فاز ۲ دادیم گفته شد یعنی پارامترهای پیش فرض فقط در بعضی شرایط خوب بودند، پس بهینه‌سازی باید روی چند سناریو باهم انجام بشه تا نتیجه قوی و کلی باشه، نه فقط برازش شده روی یک حالت خاص.

در بخش دوم فایل ما یک نقطه ی مرجع در نظر میگیریم که البته طبق experiment هایی که داشتیم نقطه ی مناسبی در فضای مسئله میباشد. اصطلاحا این نقطه را از این به بعد baseline خطاب میکنیم و در پلات ها هم با همین اسم مشاهده خواهد شد.

بخش سوم فایل میشود چهار معیار مقایسه مان.

معیار ۱ — مقدار نهایی تابع هزینه

هر دو الگوریتم یک بار اصلی (headline run) اجرا میشن، و بهترین هزینه‌ی نهایی‌شون با هم و با baseline مقایسه میشه.

معیار ۲ — سرعت همگرایی

تاریخچه‌ی "بهترین هزینه تا این لحظه" برای هر دو الگوریتم در طول تکرارها ذخیره میشه و به‌صورت یک نمودار همگرایی (pso_vs_aco_convergence.png) رسم میشه — تا ببینیم کدوم الگوریتم سریع‌تر به جواب خوب می‌رسه.

معیار ۳ — پایداری راه‌حل در اجراهای مختلف

اینجا هر دو الگوریتم دوباره، این بار با ۵ تا seed مختلف (STABILITY_SEEDS = [1,2,3,4,5]) اجرا میشن — این seedها مربوط به تصادفی‌بودن خود الگوریتم بهینه‌سازی هستن (نه ترافیک). برای هر الگوریتم، میانگین و انحراف معیار هزینه‌ی نهایی، و همچنین میانگین انحراف معیار خود پارامترها (نه فقط هزینه) محاسبه میشه. هر چی این اعداد کمتر باشن، یعنی الگوریتم پایدارتره — به جواب مشابهی میرسد صرف‌نظر از seed اولیه.

معیار ۴ — اثر بهینه‌سازی روی عملکرد واقعی کنترلر

اینجا سه تا کنترلر (baseline, PSO, ACO-تنظیم‌شده) روی هر تک‌تک سناریوها جداگانه اجرا میشن (نه فقط میانگین کلی) — تا مطمئن بشیم بهبود کلی، یک شکست پنهان تو یک سناریوی خاص رو پنهان نکرده باشه. مقادیر خام W، Q، S هم برای هر سناریو ذخیره میشوند (نه فقط هزینه‌ی نهایی) که به‌عنوان generated data که مستندات پروژه خواسته، محسوب میشه.

اجرای دو الگوریتم با تنظیمات یکسان و مقایسه نهایی:

پس از اجرای فایل compare_optimizers.py به نتایج زیر میرسیم:

هر دو الگوریتم تحت شرایط کاملاً یکسان ارزیابی شده‌اند:

- ۴ سناریوی مختلف ترافیکی
- ۳ بذر تصادفی
- ۱۰۰۰ گام شبیه‌سازی برای هر اجرا
- ۱۰۰ تکرار بهینه‌سازی

PSO	ACO
73.6823	73.6823
68.1014	72.5857
5.5809	1.0966
7.75%	1.49%
100	100

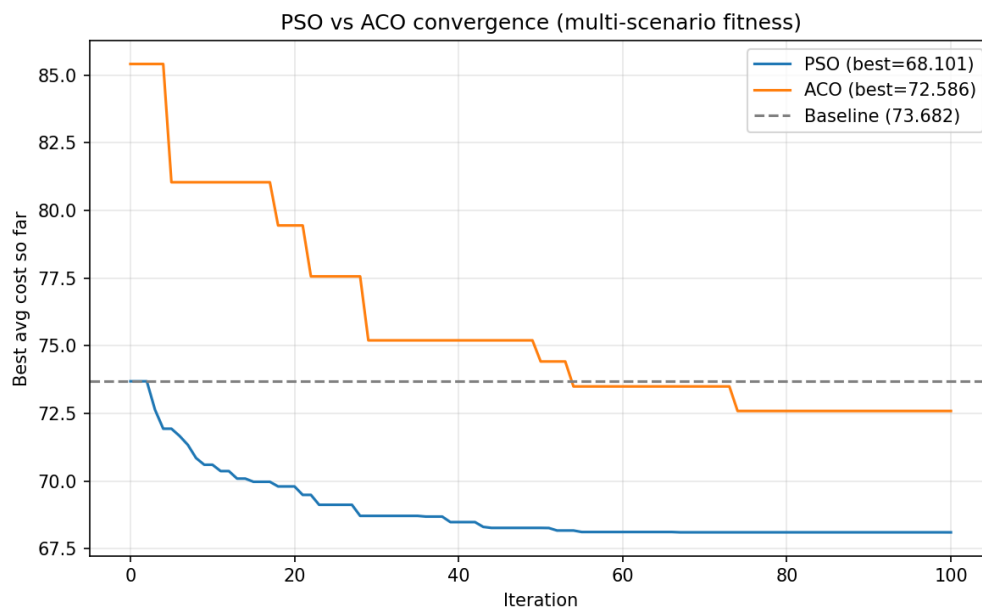
همان‌طور که مشاهده می‌شود، الگوریتم PSO مقدار تابع هزینه را از 73.68 به 68.10 کاهش داده است، در حالی که الگوریتم ACO تنها آن را به 72.59 رسانده است. بنابراین PSO حدود 7.57 درصد نسبت به کنترل‌کننده اولیه بهبود ایجاد کرده، در حالی که این مقدار برای ACO تنها 1.49 درصد است.

اگر این دو الگوریتم مستقیماً با یکدیگر مقایسه شوند:

- اختلاف مقدار Cost برابر است با

$$72.5857 - 68.1014 = 4.4843$$

که نشان می‌دهد PSO مقدار تابع هزینه را حدود 6.18 درصد کمتر از ACO کرده است.



مقادیر پارامترهای بهینه‌شده کنترل‌کننده فازی:

کنترل‌کننده فازی شامل ۲۷ پارامتر قابل تنظیم است. از این تعداد، ۱۸ پارامتر مربوط به توابع عضویت ورودی و خروجی (سه تابع عضویت با سه نقطه (a, b, c) و ۹ پارامتر مربوط به وزن قوانین فازی هستند.

۱. پارامترهای توابع عضویت صف :

حالت پایه (Baseline)

پس از بهینه‌سازی با PSO

تابع عضویت	a	b	c
Low	0.0	0.0	5.5885
Medium	0.0	7.4238	18.4083
High	16.7957	20.0	20.0

پس از بهینه‌سازی با ACO

تابع عضویت	a	b	c
Low	1.5	1.5	17.5

تابع عضویت	a	b	c
Medium	0.5	3.5	15.5
High	12.5	14.5	14.5

2. پارامترهای توابع عضویت زمان چراغ سبز

حالت پایه (Baseline)

تابع عضویت	a	b	c
Short	5.0	5.0	25.0
Medium	11.0	25.0	39.0
Long	25.0	45.0	45.0

پس از بهینه‌سازی با PSO

تابع عضویت	a	b	c
Short	5.0	5.0	15.0036
Medium	5.0	9.1123	36.1878
Long	23.9845	43.5850	45.0

پس از بهینه‌سازی با ACO

تابع عضویت	a	b	c
Short	6.0	6.0	34.0
Medium	22.0	34.0	40.0
Long	22.0	26.0	44.0

3 وزن قوانین فازی

قانون فازی	Baseline	PSO	ACO
اگر صف 1 Low و صف 2 Low، آنگاه زمان سبز Medium	1.0000	0.9768	0.375
اگر صف 1 Low و صف 2 Medium، آنگاه زمان سبز Short	1.0000	0.9491	0.775
اگر صف 1 Low و صف 2 High، آنگاه زمان سبز Short	1.0000	0.9383	0.625
اگر صف 1 Medium و صف 2 Low، آنگاه زمان سبز Long	1.0000	0.9980	0.225
اگر صف 1 Medium و صف 2 Medium، آنگاه زمان سبز Medium	1.0000	1.0000	0.075
اگر صف 1 Medium و صف 2 High، آنگاه زمان سبز Short	1.0000	0.9860	0.875
اگر صف 1 High و صف 2 Low، آنگاه زمان سبز Long	1.0000	1.0000	0.375
اگر صف 1 High و صف 2 Medium، آنگاه زمان سبز Long	1.0000	0.9978	0.175
اگر صف 1 High و صف 2 High، آنگاه زمان سبز Medium	1.0000	1.0000	0.125